Report of Assignment 1, Foundation of Machine Learning (DA5400)

Harshil Pandey, DA24C006

September 20, 2024

1 Details of the Submission

- `models.py` file contains all the source code used to implement the solutions to the questions.
- `demonstration.py` file uses the models implemented in `models.py` to demonstrate and arrive at the solution. To run `demonstration.py`, keep the `models.py` in the same directory and change the `train_path` and `test_pat` variables defined in the `demonstration.py` file to the corresponding path of the training and testing data.
- Libraries Used:
  - `numpy`: for matrix and numerical operations
  - `pandas`: for loading and storing data in matrix format.
  - `matplotlib`: for plotting the graphs
  - `seaborn`: for plotting the graphs
- The measure of error used is the Mean Squared Error explained later.

2 Details of the Source Code

The theory discussed in the below subsections (except `KernelRegression` subsection) that refers to X as the data matrix, assumes that the bias column has been added. However, none of the methods that accept the '`X`' parameter assume that. In all such methods, the intercept column is added internally. So while discussing the theoretical aspects the reader should assume that the bias column is added (except `KernelRegression` subsection), but while calling the methods, the data matrix should not have the bias column as the methods take care of it.

- `mse(y_true, y_pred)`: This function calculates the Mean Squared Error of the predictions.
  - `y_true` parameter is the true output variable.
  - `y_pred` parameter is the predicted output variable.

- Both parameters are expected to be `numpy arrays`.
- The formula is $\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$. Where y_i and $\hat{y}_i$ represent the true output variable and predicted output variable for the i^{th} data point respectively.

- `OLS`: This class implements the analytical solution of Linear Regression.
  - The hypothesis of this method is given by $y = \sum_{j=1}^{d} w_j x_j + w_0$. Where w_j is the weight associated with the j^{th} feature and w_0 is the intercept value. The goal is to find the best weights such that the predictions ($\hat{y}$) calculated using these weights produce the minimum Mean Squared Error.
  - `fit(X, y)` method calculates the weights (w_js). `X` is the data matrix that has data points as rows and features as columns, expected to be a `numpy ndarray` or a `pandas dataframe`. `y` is the output variable which is expected to be a `numpy array` or a `pandas series`.
  - The weights are calculated and stored in the `coef_` variable of the class. The last entry in `coef_` is the intercept value. The formula to calculate the weights is given by:
    $$w_{ML} = (X^T X)^{-1} X^T y.$$
  - `predict(X)` method calculates the predictions for data stored in `X`. `X` is the data matrix that has data points as rows and features as columns, expected to be a `numpy ndarray` or a `pandas dataframe`.
  - The predictions are calculated by the formula: Xw_{ML}

- `GradientDescent`: This class implements the gradient descent solution to Linear Regression.
  - The class expects `step_size` and `iterations` from the user that corresponds to the gradient step size and total iterations to be performed. Similar to `OLS` class, the weights are stored in the `coef_` variable of the class.
  - `gradient(X, y)` method calculates the gradient based on the data and current weights. The requirements for `X` and `y` are the same as in `OLS`. The formula of the gradient is given by: $\nabla f(w) = X^T X w - X^T y$.
  - The update rule is given by : $w_{t+1} = w_t - \eta \nabla f(w_t)$. Where η is the step size.
  - `fit(X, y, w_ml)` implements the gradient descent algorithm as given by the above update rule. The requirements for `X` and `y` are the same as in `OLS`. Along with this, this method also finds the error between w_{ML} and w_t for each iteration, where w_{ML} represents the solution given by `OLS` method. The errors are calculated as $||w_{ML} - w_t||_2$ and stored in `errors` variable of the class.

- `predict(X)` method calculates the predictions similar to the `OLS` class. The predictions are calculated as $X\hat{w}$, where $\hat{w}$ is the weight vector obtained after running the gradient descent algorithm.

- `StochasticGradientDescent`: This class implements the stochastic gradient descent method for Linear Regression. The batch size is taken as 100 data points at a time. The class design is the same as `GradientDescent` class. The only difference lies in the `fit(X, y, w_ml)` method.
  `fit(X, y, w_ml)` implements the technique and calculates the weights. Instead of taking the whole dataset for the gradient update rule, it takes a random batch of 100 data points to calculate the gradient. The program is such that it ensures that each data point is selected in the training batch at least once. Similar to `GradientDescent` class, this method also stores the error between w_{ML} and w_t for each iteration in the `errors` variable. After completing the iterations, the final weights are stored as the average of all the intermediate weights learned in each iteration. The formula is given by: $\hat{w} = \frac{1}{T}\sum_{t=1}^{T} w_t$.

- `RidgeGD`: This class implements the Ridge Gradient Descent Algorithm. The class design is very similar to `GradientDescent` class, some key differences are highlighted below:

  - In addition to `step_size` and `iterations` this class also expects the `regu` parameter which corresponds to the regularization parameter in Ridge Regression.
  - `gradient(X, y)` method calculates the gradient of the objective function of the Ridge formulation. Hence the formula is different as compared to `GradientDescent` and given by:

    $$\nabla f(w) = X^T X w - X^T y + \lambda w$$

    The update rule is the same as `GradientDescent`.

- `KernelRegression`: This class implements the Polynomial Kernel Regression algorithm.

  - The class expects a `degree` parameter that specifies the degree of the kernel function. The class also stores the data passed during the `fit(X, y)` method in the `data` variable, as it will be required for calculating the Kernel Matrix. `X` and `y` have the same requirements as in `OLS`.
  - The formula for calculating the kernel function for two vectors is given by: $k(x_1, x_2) = (x_1^T x_2 + 1)^p$ where p is the degree specified by the user. This value is the same as taking dot product in higher dimension hence, $k(x_1, x_2) = \phi(x_1)^T \phi(x_2)$ where $\phi(x)$ represents the function that transforms x to a higher dimension.
  - Instead of calculating the optimal weights in the higher dimension, we calculate coefficients of combinations (given by α) of data points that lead us to the optimal weights. So $\hat{w} = \phi(X)^T \alpha$

- The formula for calculating α is given by: $\alpha = K^{-1}y$, where K is the kernel matrix of the training data which is in turn calculated as $K_{ij} = k(x_i, x_j)$ where x_i and x_j are the i^{th} and j^{th} data points of the data matrix.
- `calc_kernel(X_new)` method calculates the kernel matrix corresponding to `X_new` which represents a data matrix similar to `X` that was passed to `fit(X, y)` method. The formula for calculating the kernel matrix is given by: $K_{ij}^{new} = (< X_i^{new}, X_j > +1)^p = k(X_i^{new}, X_j)$, where $< X_i^{new}, X_j >$ is the dot product between the i^{th} data point (row) of X^{new}, and j^{th} data point (row) of X (data matrix on which the model has been trained). In `numpy` the whole matrix is calculated easily by `(X_new @ X.T + 1)**p`. This method is also used to calculate K.
- The prediction is made using α and K^{new} with the formula: $K^{new}\alpha$
- `fit(X, y)` method calculates the α.
- `predict(X)` method calculates the predictions by calculating K^{new} and using the already calculated α.

- `generate_splits(n, test_size)`: This method generates integer indices for training and testing subsets of data of size `n` with `test_size` representing the fraction of data points that go into the testing subset. This method is used in the cross-validation procedure to generate splits for data. This method returns an iterator instead of returning all the splits at once. The method ensures that each data point goes into the testing set only once. The number of splits is determined accordingly.

- `cross_validate(X, y, model, test_size)`: This method implements the cross validation process. `X` and `y` represent the data matrix and output vector with the usual requirement as in previous methods.
  - `model` is a parameter that accepts the model that needs to be cross-validated for example an `OLS` or `RidgeGD` instance.
  - `test_size` represents what fraction of data points go into each validation subset.
  - The error(MSE) on the validation subset for each iteration is calculated and the average of the errors is returned.

3 Observations and Inferences

3.1 Analytical Solution

The weights corresponding to features given by the analytical solution are as follows: $w_{ML} = (1.766,\ 3.522,\ 9.894)$
the first, second, and third component is the weight corresponding to the first, second, and third (intercept added by algorithm) features of the data respectively. Hence the function learned is: $y = 1.766x_1 + 3.522x_2 + 9.894$

3.2 Gradient Descent Solution

The algorithm was run for 200 iterations with a constant step size of 0.0001. The weights corresponding to features given by gradient descent are as follows:

$$w_{GD} = (1.766,\ 3.522,\ 9.894)$$

We observe that the solution is the same as that of the analytical method.
Figure 1 shows a graph that plots the error between w_{ML} and w_t as a function of t. Here, w_t represents the weights obtained in t_{th} iteration and the error is calculated as: $||w_{ML} - w_t||_2$
It is observed that as the iterations increase, the error between w_{ML} and w_t

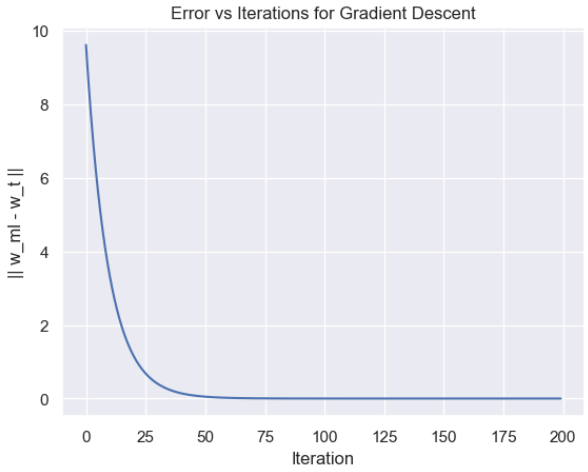

Figure 1: Convergence of Gradient Descent

reduces smoothly. The reduction is much higher in the earlier iterations and after 100 iterations the error is very close to 0 and the reduction is negligible. This can be attributed to the fact that as w_t becomes closer and closer to w_{ML}, the gradient ($\nabla f(w_t)$) approaches to zero vector, due to which the change caused in w_t according to the gradient descent update rule is negligible.

3.3 Stochastic Gradient Descent Solution

The algorithm was run for 10000 iterations with a step size that depends on the iteration number, the formula for the same is:

$$\eta = \frac{1}{100t}$$

This was done to ensure that in later iterations when w_t is closer to w_{ML}, the algorithm makes smaller updates. This reduces the chance of diverging from w_{ML} due to a bad sample. 100 is added in the denominator to keep the step size below 0.01 in the initial iterations, otherwise, larger step sizes (1, 0.5, 0.25, etc) may cause the weights to diverge from w_{ML}. This was not needed in the normal gradient descent case as sampling was not done and a constant step size of 0.0001 did the job. The weights corresponding to features given by stochastic gradient descent are as follows:

$$w_{SGD} = (1.766,\ 3.522,\ 9.894)$$

The solution is the same as the solution of the analytical method. From *Figure*

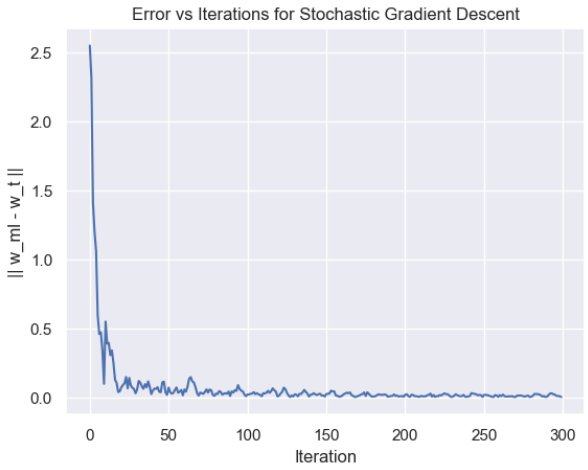

Figure 2: Convergence of Stochastic Gradient Descent

2 it is evident that the error does not reduce smoothly and in fact, increases many times randomly as the iterations proceed. When compared to the plot for gradient descent, we can find a similarity in the early iterations that the error reduces sharply but the difference is vivid for further iterations. This is because, in Stochastic Gradient Descent, we make an update to the weights just by considering a random sample of the data and sometimes it may happen that the direction of the gradient obtained from the sampled data may not be a descent direction, resulting in a jagged curve. Another difference is that the Stochastic Gradient Descent required 10000 iterations to converge to the solution as compared to just 200 iterations in the Gradient Descent.

3.4 Ridge Gradient Descent Solution

The Ridge Gradient Descent Algorithm was cross-validated for all integer values of λ (regularization parameter) in the interval $[0, 40)$. The cross-validation model for each iteration was run for 1000 iterations and with a step size of 0.0001. The cross-validation error was calculated and *Figure 3* highlights the error as a function of λ. It is observed that the error marginally reduces as we move from $\lambda = 0$ to $\lambda = 5$ and then continuously increases as we increase λ further. $\lambda = 0$ corresponds to normal gradient descent. The minimum error is obtained when $\lambda = 3$. The weights corresponding to features given by ridge gradient descent when $\lambda = 3$ are as follows:

$$w_{RGD} = (1.759,\ 3.512,\ 9.865)$$

Let's compare the performance of the solutions obtained by the Analytical Method and Ridge Gradient Descent by calculating the error on the test data:

$$w_{ML} = (1.766,\ 3.522,\ 9.894)$$

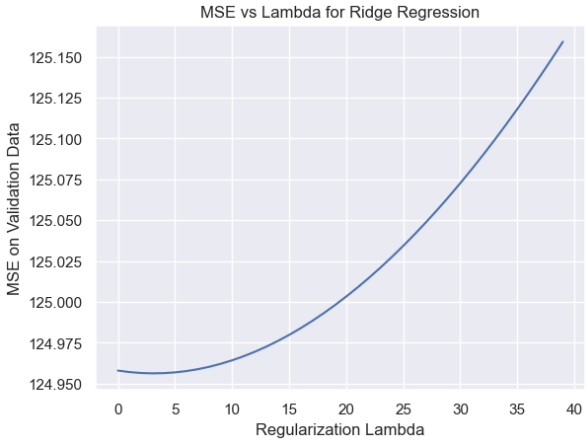

Figure 3: Cross-Validation of Ridge Gradient Descent

$$OLS\ Test\ Error:\ 66.005$$

$$w_{RGD} = (1.759,\ 3.512,\ 9.865)$$
$$Ridge\ Test\ Error:\ 65.960$$

We can see that the Ridge solution results in a smaller error for test data. Since both the cross-validation error and test error are lower for the Ridge solution, we can conclude that the Ridge solution is better than the Analytical Solution but not very significantly.

3.5 Kernel Regression Solution

Before going into the actual solution, let us visualize the relationship of the features with the output variable. This is easily done through a scatter plot which *Figure 4* shows.

We can see that both features are non-linearly related to the output variable and both scatter plots seem to show parabolic curvature. This brings us to the use of polynomial kernel methods to incorporate these relationships in our analysis. Kernels allow us to perform analysis in higher dimensions where non-linear relationships become linear without increasing much computational complexity of the method.

Cross-validation was used to find the best degree for the kernel method and it was found to be `degree=2`, as we expected from the scatter plots. *Figure 5* shows the cross-validation error as a function of the degree of the kernel applied. `degree=1` corresponds to the standard least squares regression (OLS) method. We observe that the error goes to approximately 0 when the degree is increased from 1 to 2. Let's compare the performance of the solutions obtained by the Analytical Method and Kernel Regression (degree=2) by calculating the error on the test data:

$$OLS\ Test\ Error:\ 66.005$$

$$Kernel\ Test\ Error:\ 0.010$$

Figure 4: Scatter Plot of Features and Output

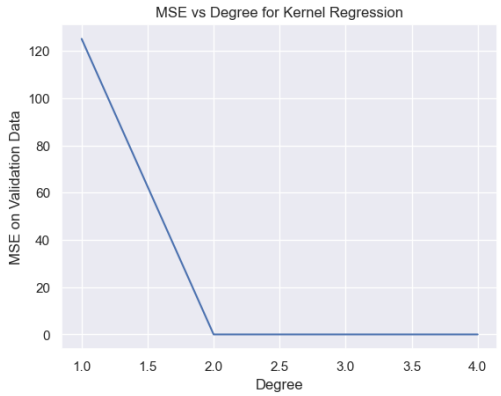

Figure 5: Cross-Validation of Kernel Regression

We can see that the Kernel solution results in negligible error for test data. This is because the Kernel method can capture non-linear relationships, in our case, second-order relationships between the output and the features which the OLS method can't. Hence, in this case, the Kernel method is significantly better than the OLS method. This fact is also backed by the notably lower cross-validation error and testing error of the Kernel solution when compared to the OLS solution.